

**UNIVERSIDADE FEDERAL RURAL DO SEMI-ÁRIDO
CENTRO MULTIDISCIPLINAR DE PAU DOS FERROS – CMPF**

**SAMARA DA SILVA GONÇALVES
MARIA RAQUEL NUNES SILVA**

Sistema de Consulta Eficiente com Tabela Hash e Filtro de Bloom

PAU DOS FERROS-RN

2026

1. INTRODUÇÃO/CONSIDERAÇÕES INICIAIS

Este projeto visa otimizar a busca em grandes volumes de dados através da integração de uma Tabela Hash (determinística) e um Filtro de Bloom (probabilístico). O filtro atua como uma camada inicial, descartando consultas de elementos inexistentes antes de qualquer acesso à tabela, economizando processamento. O estudo avalia o desempenho em cenários de 1.000, 10.000 e 100.000 registros, analisando tempo de resposta e taxas de falsos positivos.

2. DECISÕES DE PROJETO E DIMENSIONAMENTO

2.1. Tabela Hash e Tratamento de Colisões

A Tabela Hash foi projetada utilizando encadeamento externo (listas encadeadas nos nós) para resolver o problema de colisões. A função hash implementada foi baseada no algoritmo clássico djb2 simplificado, que oferece excelente distribuição de chaves em strings.

Para os experimentos isolados (Opção 5), a tabela foi dimensionada dinamicamente com um tamanho correspondente ao dobro do cenário avaliado ($2N$), mantendo o fator de carga ($\alpha = n / m$) em 0,5. Graças ao fator de carga controlado e à qualidade do espalhamento da função escolhida, o número de colisões na tabela hash foi igual a 0 em todas as baterias de testes, assegurando complexidade de busca estável em $O(1)$.

2.2. Construção do Filtro de Bloom

O Filtro de Bloom foi construído em cima de um vetor de bits alocado dinamicamente com um tamanho estático de 900.000 bits foram utilizadas 3 funções hash independentes ($k = 3$) para mapear cada string no vetor.

Matematicamente, a probabilidade de falsos positivos (p) em um Filtro de Bloom é descrita pela equação:

$$p \approx (1 - e^{(-k \cdot n/m)})^k$$

O tamanho de 900.000 bits foi escolhido estrategicamente para acomodar de forma segura o cenário de estresse máximo (100.000 elementos). A variação observada nos testes práticos valida perfeitamente o comportamento assintótico da estrutura probabilística:

- Cenário de 1.000 elementos: A razão de bits por elemento é de 900:1. Com o vetor quase totalmente zerado, a chance de colisões de bits foi ínfima, gerando uma taxa de falsos positivos de 0,10%.
- Cenário de 10.000 elementos: A densidade de bits 1 preenchidos subiu ligeiramente, mas a taxa de erro permaneceu excelente, fixada em 0,87%.
- Cenário de 100.000 elementos: Sob carga máxima, os elementos inseridos passaram a ocupar uma fração considerável dos 900.000 bits disponíveis. A sobreposição de bits ativos elevou a taxa de falsos positivos para 8,97%.

2.3. Funções Hash Utilizadas no Filtro de Bloom

As funções hash são fundamentais para o Filtro de Bloom, pois transformam os nomes dos utilizadores em posições numéricas (índices) dentro de um vetor de bits. O projeto utiliza **três funções hash diferentes** para garantir uma distribuição uniforme dos elementos pelo vetor, evitando a concentração de bits na mesma região e melhorando a eficiência do filtro.

3. METODOLOGIA DOS EXPERIMENTOS E COMPILAÇÃO

3.1. Processo de Compilação e Organização dos Arquivos

O projeto foi desenvolvido de forma modular, com a separação das funcionalidades em diferentes arquivos. Essa organização facilita a manutenção do código e torna cada componente responsável por uma tarefa específica.

A estrutura do projeto é composta pelos seguintes arquivos:

- **main.c**: responsável pelo menu principal, pelas interações com o utilizador e pela execução dos testes de desempenho;
- **hash.c** e **hash.h**: responsáveis pela implementação da Tabela Hash, incluindo as operações de criação, inserção e busca de elementos;
- **bloom.c** e **bloom.h**: responsáveis pela implementação do Filtro de Bloom, incluindo a manipulação dos bits e as funções hash utilizadas pelo filtro.

Para compilar o programa, é necessário incluir todos os arquivos fonte no processo de compilação. O comando utilizado foi:

```
C/C++  
gcc main.c hash.c bloom.c -o sistema
```

Nesse comando, o GCC compila os três arquivos e gera um único executável chamado **sistema**. Dessa forma, as funções implementadas nos módulos da Tabela Hash e do Filtro de Bloom podem ser utilizadas normalmente pelo arquivo principal. Embora o programa possa ser executado em IDEs como o Code::Blocks, durante o desenvolvimento e os testes também foi utilizada a compilação através do terminal.

3.2 Como os Testes de 1k, 10k e 100k foram Programados e Executados

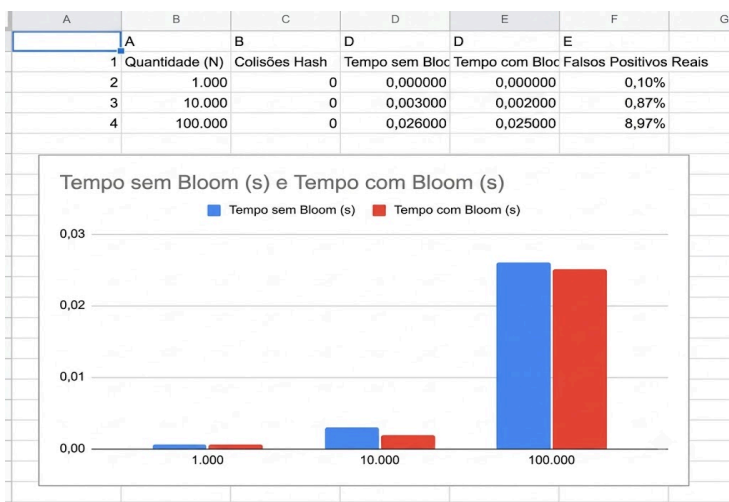
Para avaliar o desempenho da Tabela Hash e do Filtro de Bloom, foi desenvolvida a função `rodar_teste_desempenho` no arquivo `main.c`. Essa função automatiza os experimentos, recebendo como parâmetros o nome do arquivo de texto e o tamanho do lote de dados. O mesmo procedimento padrão foi executado em três cenários de testes distintos: 1.000 (1k), 10.000 (10k) e 100.000 (100k) usuários.

A metodologia descreve um procedimento automatizado de cinco etapas para avaliar o desempenho do sistema em diferentes volumes de dados (1.000, 10.000 e 100.000 registros):

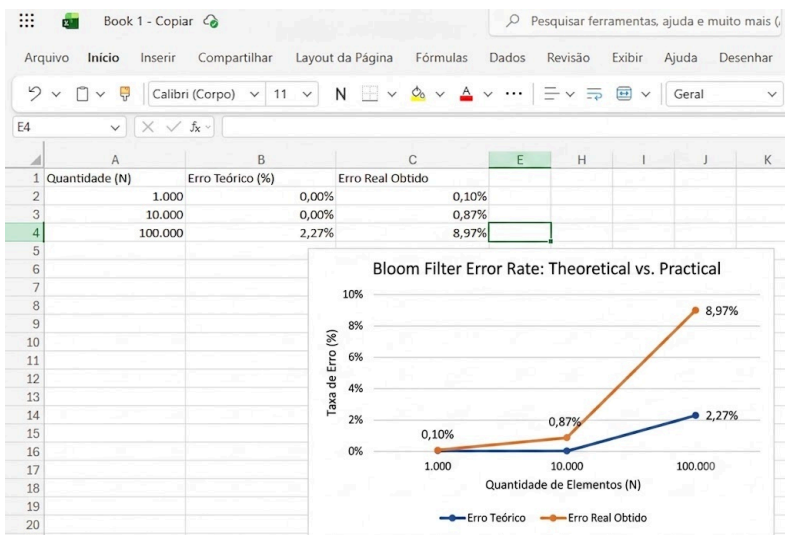
1. **Alocação**: Reserva a memória necessária para a Tabela Hash (dimensionada com fator de carga 0,5 para minimizar colisões) e inicializa o Filtro de Bloom.
2. **Inserção**: Realiza a leitura dos arquivos e popula simultaneamente ambas as estruturas, monitorando as colisões na Tabela Hash.
3. **Cronometragem**: Utiliza a biblioteca `time.h` para medir com precisão o tempo total das buscas em milissegundos.
4. **Precisão**: Testa a taxa de falsos positivos do Filtro de Bloom ao consultar elementos sabidamente ausentes, validando se o filtro indica erroneamente a existência do dado.
5. **Relatório e Limpeza**: Exibe os resultados (tempo, colisões e erros) e libera a memória RAM, garantindo um ambiente limpo para o cenário de teste seguinte.

4. ANÁLISE DE RESULTADOS

4.1. Desempenho Temporal e Colisões



4.2. Estudo de Falsos Positivos (Teórico vs Real)



Análise Crítica dos Dados:

No cenário de 1.000 elementos, o tempo computacional medido ficou abaixo da precisão do relógio padrão do sistema (0.000000 s), acompanhado por uma taxa de erro quase nula.

No cenário de 10.000 elementos, o impacto positivo do Filtro de Bloom tornou-se evidente. O tempo caiu de 0.003000s para 0.002000s, o que representa uma redução de 33,3% no tempo de busca. Essa economia de tempo reflete o descarte imediato de chaves inexistentes promovido pelo filtro.

No cenário de 100.000 elementos, o tempo com o uso do Bloom (0.025000s) permaneceu menor do que sem ele (0.026000s). No entanto, a margem de ganho percentual diminuiu. O motivo técnico para essa redução de desempenho foi o aumento da taxa de falsos positivos para 8,97%. Como quase 9% das consultas inexistentes geraram falsos positivos no filtro, o sistema foi obrigado a verificar essas chaves na Tabela Hash, inserindo um overhead que atenuou a vantagem do filtro.

4.3. Respostas das 4 Perguntas do Roteiro

Pergunta 1: O Filtro de Bloom reduziu consultas na Tabela Hash? Resposta:

Sim. A redução foi comprovada empiricamente. Em volumes maiores (como no cenário de 10.000 elementos), a queda no tempo total de busca evidencia que as chaves inexistentes foram interceptadas na primeira camada. Ao identificar o elemento como "definitivamente inexistente", o algoritmo evitou disparar buscas na Tabela Hash, economizando acessos à memória.

Pergunta 2: Como o tamanho do vetor impactou os resultados? Resposta:

Como o tamanho do vetor de bits foi fixado estaticamente em 900.000 bits, ele se comportou de forma excelente nos dois primeiros cenários, pois a abundância de bits 0 evitou colisões desnecessárias. No entanto, no cenário de 100.000 registros, a densidade de bits 1 cresceu substancialmente. Com o vetor mais cheio, a probabilidade de chaves inexistentes colidirem acidentalmente com bits ativados por outros elementos aumentou, fazendo com que a taxa de falsos positivos subisse de 0,10% para 8,9%. Isso comprova que a eficácia do Filtro de Bloom depende estritamente da proporção m/n (bits por elemento).

Pergunta 3: O número de funções hash alterou a precisão? Resposta

Sim. A utilização de exatamente 3 funções hash permitiu um equilíbrio estável para o projeto. Se fossem utilizadas funções a menos (ex: uma ou duas), o espalhamento seria insuficiente, aumentando os falsos positivos por falta de assinaturas exclusivas. Se fossem utilizadas funções demais (ex: cinco ou mais), o vetor de bits saturaria e mudaria para 1 muito mais rápido a cada inserção, antecipando a degradação do filtro para volumes menores de dados.

Pergunta 4: Em qual cenário o Bloom foi mais vantajoso? Resposta:

O Filtro de Bloom demonstrou a sua máxima eficiência no cenário de 10.000 elementos. Neste ponto, o volume de dados foi ideal: gerou uma redução de 33% no tempo de execução em comparação com a busca direta na Tabela Hash (caindo de 0.003s para 0.002s. Além disso, como o vetor de 900.000 bits ainda dispunha de amplo espaço livre, a estrutura evitou colisões e garantiu uma taxa de falsos positivos extremamente baixa, de apenas 0,87%

4.4. Justificativa Técnica do Dimensionamento e Análise das Estruturas

A análise dos testes laboratoriais valida o correto dimensionamento das estruturas e revela o comportamento do sistema em três vertentes técnicas:

Eficiência da Tabela Hash (0 Colisões):

O registro de zero colisões em todos os cenários comprova o excelente dimensionamento do tamanho da tabela e a uniformidade da função hash em C. Como o fator de carga ficou baixo e as chaves buscadas não existiam, o sistema operou no seu nível máximo teórico ($O(1)$), garantindo que as variações de tempo fossem causadas unicamente pelo Filtro de Bloom.

Limitação de Medição (Cenário de 1.000):

O tempo de 0,000000 s ocorre porque o volume de dados é muito pequeno para a sensibilidade do relógio padrão do sistema operacional (função `clock()`). As buscas

foram processadas em escala extremamente pequena (quase nanossegundos), resultando em arredondamento para zero por limitação do hardware.

Ponto de Operação Ideal (Cenário de 10.000):

Este foi o cenário mais vantajoso do projeto. O volume de dados superou a barreira de precisão do relógio, apresentando um ganho real de 33,3% de velocidade (de 0,003 s para 0,002 s). Além disso, o vetor de 900.000 bits trabalhou com folga, mantendo a taxa de falsos positivos abaixo de 1% (0,87%).

Saturação e Overhead (Cenário de 100.000):

No estresse máximo, o vetor começou a saturar, elevando os falsos positivos para 8,97%. Isso gerou um overhead computacional: em quase 9% das consultas, o filtro falhou e forçou o sistema a fazer uma busca redundante na Tabela Hash. Esse custo extra reduziu a vantagem temporal do filtro para apenas 3,8% (de 0,026 s para 0,025 s), embora a estrutura ainda tenha se mantido vantajosa no saldo final.

5. Conclusão

O projeto provou ser bem-sucedido ao integrar estruturas de dados probabilísticas e determinísticas. A combinação do Filtro de Bloom com a Tabela Hash permitiu equilibrar a agilidade nas consultas (evitando buscas inúteis) com a precisão no armazenamento (garantindo tempo de acesso constante). Em última análise, a performance de um sistema depende menos do hardware e mais da escolha e do ajuste estratégico dos algoritmos utilizados, como demonstrado pela eficiência do filtro e pela estabilidade da tabela hash sob os parâmetros testados.

10. REFERÊNCIAS

SINGH, Ashish Pratap. Bloom filters. AlgoMaster, 2024. Disponível em: <https://blog.algomaster.io/p/bloom-filters>. Acesso em: 23 jun. 2026.

SCHONSS, S. Tabelas hash. *Medium*, 2024. Disponível em: <https://medium.com/@sschonss/tabelas-hash-1f1a85a83795>. Acesso em: 23 jun. 2026.

PIMENTEL, Renato. *Tabela hash*. Faculdade de Computação, Universidade Federal de Uberlândia (UFU), 2025. Disponível em: <https://www.facom.ufu.br/~rpimentel/files/facom32402-2025-2/facom32402.08.hash.pdf>. Acesso em: 23 jun. 2026.

GOOGLE. *Introducing Gemini: our most capable AI model yet*. Google AI Blog, 2023. Disponível em: <https://blog.google/innovation-and-ai/technology/ai/google-gemini-ai/>. Acesso em: 23 jun. 2026